

A Few Things You Should Know About Futexes

This article aims to provide a clear and concise discussion about the basics of futexes, and how they could be used to implement user-space synchronisation mechanisms on Linux.

So what are futexes? They are a kernel mechanism (in the form of a system call), using which fast user-space synchronisation can be implemented. The kernel interface for futexes looks like what is shown below:

```
int futex(int *uaddr, int op, int val, const struct timespec
*timeout,
         int *uaddr2, int val3);
```

The second param, *op*, can have several values, of which the following two are relevant for this discussion:

- **FUTEX_WAIT:** The kernel checks if the value at *uaddr* is the same as *val*; if so, it then blocks the calling thread/process. If not, it then returns with **EWouldBlock**. The last two parameters are ignored for this *op*, and the *timeout* parameter is not relevant, at least for now.
- **FUTEX_WAKE:** The kernel wakes up at most *val* processes waiting on this futex. The last three parameters are ignored for this operation.

So why do we need futexes? Take a look at the traditional user-space synchronisation mechanisms:

- **Spin lock** involves spinning on a variable in user space if the lock is not available. No context switch to kernel space is necessary. Spin locks are great if the size of the critical section is known to be small, or if there is no alternate work to be executed after blocking the thread. A lot of research has highlighted the limitations of spin locks—not scalable with multi-cores, fairness issues, etc.

- **Semaphores** involve making a system call each time, be it contended or uncontended. Hence, they are slower.
- **Thin locks** initially try user-level locking (using atomics), similar to spin locks. If it fails, they inflate the lock and only try OS-level locking (similar to semaphores). But inflating happens only once, the first time a contention is seen. Then on, they stay inflated. Again, there's system call overhead, once the lock is inflated.

In order to address some of these issues, futexes were added to the Linux kernel to enable library developers to implement a fast user-space synchronisation mechanism. How is this done?



Note: This discussion is based on the paper 'Futexes are Tricky' by Ulrich Drepper.

Using the above *futex* system call interface, fast user-space synchronisation mechanisms can be implemented.

- 1) **The condition wait-signal mechanism:** This is typically used in producer-consumer problems, similar to what *pthread_cond_wait* and *pthread_cond_signal* provide. The following code snippet shows how this may be implemented:

```
class event {
public:
    event () : val (0) { }
    void ev_signal () {
```